

Uso de MVC como Padrão de Projeto

Objetivo da Aula

Conhecer a arquitetura MVC e aplicá-la na implementação de uma API REST, utilizando os frameworks Sequelize e Express.

Apresentação

Atualmente, a arquitetura MVC é amplamente adotada no desenvolvimento de sistemas cadastrais, pois promove uma separação entre interface de usuário, regras de negócio e persistência de dados, viabilizando a especialização dos desenvolvedores e o surgimento de ferramentas de produtividade para cada uma das áreas de interesse citadas.

Aqui, iremos compreender o que é uma arquitetura de software e aprofundar o estudo da arquitetura MVC, com sua divisão nas seguintes camadas: Model, com acesso ao banco de dados; Controller, para as operações do sistema; e View, contendo as interfaces de usuário.

Por fim, utilizaremos os frameworks Sequelize e Express para a construção de um Web Service RESTful na arquitetura MVC.

1. Padrões Arquiteturais

Quando nos referimos a **padrão arquitetural**, tratamos da estrutura geral de um sistema, segundo um padrão estabelecido para o uso de componentes e interações. Ele fornece uma visão macroscópica do sistema, permitindo observar tanto a sua relação com o ambiente externo quanto as funcionalidades internas.

Um padrão arquitetural não é um padrão de desenvolvimento, mas diversos padrões de desenvolvimento podem ser adequados para determinada estrutura de componentes.

EXEMPLO

A arquitetura **Broker**, voltada para plataformas de objetos distribuídos, utiliza os padrões **Flyweight**, para a definição do pool de objetos, e **Proxy**, adotado na comunicação remota.

Entre as características definidas por um padrão arquitetural, temos:

- Estrutura lógica do sistema;
- Componentes e interações entre eles;
- Modularidade do sistema;
- Separação de conceitos.

Um bom padrão arquitetural deve garantir que o sistema tenha qualidade, flexibilidade, escalabilidade e, em alguns casos, interoperabilidade. Embora nem todas as arquiteturas visem à interoperabilidade, essa é uma necessidade cada vez mais comum no ambiente heterogêneo da internet.

Para compreender melhor o que é um padrão arquitetural, vamos conhecer algumas arquiteturas comuns no mercado de desenvolvimento. Não é raro que um sistema combine as características de mais de uma arquitetura, como um sistema Java Web, na arquitetura MVC, utilizando componentes EJB (Broker) e serviços de mensageria (Event Driven).

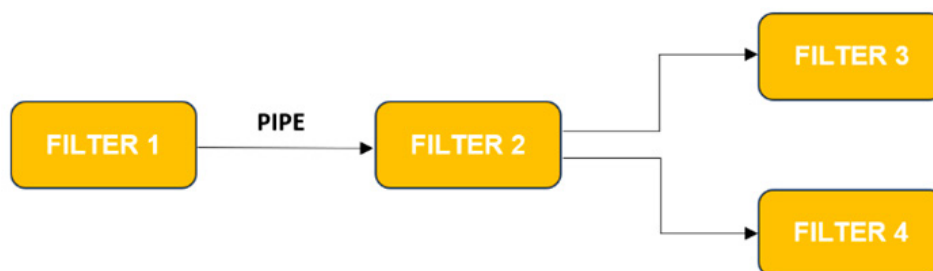
1.1. Broker

Arquitetura adotada pelas plataformas de objetos distribuídos, como CORBA, EJB e DCOM. Seu objetivo é a construção de um ambiente onde objetos de servidores diferentes possam se comunicar de forma transparente. Necessita de pools de objetos, clientes remotos e um componente **Broker**, responsável por orquestrar a comunicação entre objetos distribuídos.

1.2. Pipes and Filters

Voltada para o processamento sequencial de um fluxo de informação, os componentes da arquitetura são chamados de filtros ou **filters**, e as saídas do componente alimentam as entradas de outros via conectores ou **pipes**.

Figura 1: Arquitetura Pipes and Filters



Fonte: Elaboração própria.

1.3. Cliente e Servidor

Na arquitetura cliente e servidor, temos um componente **servidor**, capaz de oferecer algum tipo de recurso, e um componente **cliente**, que se comunica com o servidor, de forma especializada, para consumir o recurso. Os protocolos mais comuns da internet utilizam a arquitetura cliente e servidor, como FTP, SMTP, POP e HTTP, apenas para citar alguns exemplos.

1.4. Event Driven

Uma arquitetura orientada a eventos, ou **event driven**, utiliza a comunicação assíncrona entre componentes, sendo a base para os sistemas de mensagerias, como JBossMQ e RabbitMQ. Elas podem trabalhar no domínio **Point-to-Point**, quando as mensagens são enviadas para uma fila e um consumidor as recupera sequencialmente, ou **Publish-Subscribe**, em que um tópico recebe as mensagens dos publicadores para que os assinantes as recuperem.

Figura 2: Domínios P2P e Pub/Sub para mensagerias



Fonte: Elaboração própria.

Essa é uma arquitetura muito utilizada por microserviços, que se comunicam de forma assíncrona, normalmente por meio de mensagerias.

1.5. PAC

Na arquitetura **PAC** – ou **Presentation, Abstraction e Controller** – ocorre a divisão de cada componente do sistema em três partes:

- **Presentation**, contendo a interface de usuário;
- **Abstraction**, que define o modelo de dados;
- **Controller**, responsável pelas ações do componente.

Figura 3: Arquitetura PAC



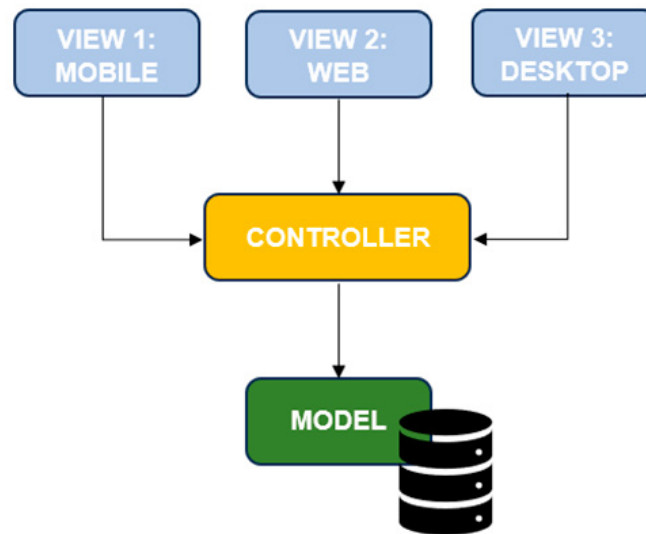
Fonte: Elaboração própria.

Qualquer comunicação entre os elementos Presentation e Abstraction deverá ser intermediada pelo Controller, e o padrão PAC se repete, ao longo de todo o sistema, para cada entidade pertencente ao seu domínio.

2. Arquitetura MVC

Atualmente, é quase obrigatória a adoção da arquitetura **MVC** em sistemas cadastrais. A sigla significa **Model, View e Controller**, fazendo referência a uma divisão do sistema em três camadas, com objetivos específicos.

Figura 4: Arquitetura MVC



Fonte: Elaboração própria.

Na camada **Model**, temos os elementos de persistência do sistema. Apenas os componentes dessa camada devem ser capazes de acessar um banco de dados, e, para todas as demais camadas, os dados serão tratados como coleções de objetos classificados como entidades.

É na camada Model que utilizamos o padrão **DAO**, ou **Data Access Object**, que concentra todas as operações relacionadas ao banco de dados para determinada entidade.

A camada **Controller** concentra as operações do sistema em objetos que são classificados como **controladores**. É nessa camada que implementamos as regras de negócio do sistema, e ela deve ser a única camada capaz de se comunicar com a Model.

Finalmente, temos a camada **View**, que representa a **interface de usuário**, com a possibilidade de múltiplas implementações para acesso à mesma camada Controller. Qualquer solicitação da View deve ser feita para um controlador, o qual deve ser neutro em relação a qualquer tipo de interface.

Em termos práticos, a regra básica é a de que qualquer comunicação entre as camadas View e Model deve ser intermediada pela Controller. Inicialmente, o padrão arquitetural MVC pode se parecer muito com o PAC, mas, para o MVC, ocorre a divisão em camadas, em que cada camada concentra um perfil de objetos, enquanto no PAC temos a divisão interna no próprio componente, com pequenos pacotes que se repetem ao longo de toda a arquitetura.

Uma grande vantagem do uso de MVC é a possibilidade de especializar o perfil do desenvolvedor para cada camada. Por exemplo, designers podem atuar na camada View, analistas de negócios na Controller e administradores de banco de dados na Model.

A especialização das camadas também permitiu a adoção de alguns padrões de desenvolvimento, e uma metodologia organizada levou ao surgimento de frameworks voltados para cada uma das camadas. Por exemplo, no ambiente do NodeJS, podemos trabalhar com Sequelize na camada Model, enquanto os Web Services RESTful, criados com Express, ficariam na camada Controller.

Quando trabalhamos com o back-end, temos apenas as camadas Model e Controller, pois o Front-end deverá ser criado posteriormente – normalmente, em outra plataforma, como Angular ou ReactJS –, configurando a camada View.

Algumas críticas são feitas à arquitetura MVC, no que se refere à quantidade de chamadas ocorrendo entre as camadas para cada ação do sistema, segundo um fluxo bidirecional. Nesse sentido, a arquitetura Flux, definida pelo Facebook, tenta resolver o problema, criando um fluxo unidirecional, em que a interface inicia ações que são despachadas e que efetuam alterações em um repositório, e este, por sua vez, fornece os dados alterados para as interfaces que o observam.

3. Camada Model com Sequelize e DAO

Para construir a camada **Model**, utilizaremos o **Sequelize** e uma classe **DAO**, que deverá concentrar as operações efetuadas para a entidade Nota. Vamos criar um projeto e repetir alguns passos anteriores.

Crie um diretório vazio, abra no VS Code e digite os comandos seguintes no terminal interno do editor.

```
npm init -y
npm install sequelize mysql2
npm install --save-dev sequelize-cli
npx sequelize-cli init
```

Através da sequência de comandos, temos a configuração do projeto para NodeJS, a instalação das bibliotecas necessárias e a criação da estrutura para uso de Sequelize no projeto. O passo seguinte é a configuração da conexão com o banco de dados, no

arquivo **config.json**, para o ambiente de desenvolvimento, como pode ser observado no fragmento seguinte.

```
"development": {  
  "username": "root", "password": "admin123",  
  "database": "agenda_v2_dev", "host": "127.0.0.1",  
  "dialect": "mysql"  
},
```

A entidade **Nota**, que será nosso elemento persistente, será gerada através do comando apresentado a seguir.

```
npx sequelize-cli model:generate --name Nota  
--attributes titulo:string,descricao:string
```

Com todos os elementos configurados, podemos criar o banco de dados e a tabela de notas, relacionada com a entidade Nota, através dos comandos que são apresentados na listagem seguinte.

```
npx sequelize-cli db:create  
npx sequelize-cli db:migrate
```

Embora o **Sequelize** já faça o **mapeamento objeto-relacional** e tenhamos todos os elementos de persistência necessários nesse ponto, o uso do padrão DAO trará maior organização ao nosso código. Crie o arquivo **notas-dao.js**, em uma pasta com o nome **daos**, e utilize o conteúdo da listagem seguinte para a definição da classe NotasDAO no arquivo.

```
const db = require('../models/index.js');  
class NotasDAO{  
  obterTodos = async() => await db.Nota.findAll();  
  obter = async(id) => await db.Nota.findByPk(id);  
  incluir = async(objJSON) => await db.Nota.create(objJSON);
```

```
alterar = async(_id, objJSON) =>
await db.Nota.update(objJSON, {where: { id: _id }});
excluir = async(_id) =>
await db.Nota.destroy({ where: { id: _id } });
}
module.exports = NotasDAO
```

Aqui, definimos a classe **NotasDAO**, com os métodos para gerenciamento de dados de notas por meio do Sequelize. Todos os métodos são assíncronos, com a marcação **async**, e as chamadas para o Sequelize são precedidas por **await**, de forma a aguardar a execução e depois retornar o resultado como **promise**.

Temos dois métodos de consulta, em que **obterTodos** retorna todas as notas do banco de dados, via método **findAll**, enquanto **obter**, que tem o **id** da nota como parâmetro, retorna apenas uma nota, com a utilização de **findByPk**.

O mesmo tipo de programação é utilizado para a manipulação de dados, a começar pelo método **incluir**, com os dados sendo fornecidos no formato **JSON** e a geração da nota no banco efetuada pelo método **create**. Já o método **excluir** recebe o **id** da nota que será excluída, sendo utilizado na restrição de execução para o método **destroy**. Finalmente, o método **alterar** recebe o **id** e os **dados** como parâmetros e invoca o método **update**, fornecendo os dados recebidos e utilizando o **id** na restrição de execução.

Nossa camada Model está pronta, e já podemos construir os controladores.

4. Camada Controller com Express

Nosso sistema será um Web Service **RESTful**, logo os controladores podem ser associados ao tratamento das requisições HTTP, em que a classe DAO será utilizada para efetuar as operações da arquitetura REST sobre o banco.

Vamos utilizar o **Express** para implementação do servidor, sendo necessário instalar as dependências através do comando apresentado a seguir.

```
npm install express body-parser
```

No passo seguinte, crie o arquivo **rota-notas.js**, utilizando o código seguinte.


```
const express = require('express');
const NotasDAO = require('./daos/NotasDAO.js');
const router = express.Router();
const dao = new NotasDAO();
router.get('/', async(req,res)=>{
  res.json(await dao.obterTodos());
})
router.get('/:id', async(req,res)=>{
  res.json(await dao.obter(req.params.id));
})
router.post('/', async(req,res)=>{
  res.json(await dao.incluir(req.body));
})
router.delete('/:id', async(req,res)=>{
  res.json(await dao.excluir(req.params.id));
})
router.put('/:id', async(req,res)=>{
  res.json(await dao.alterar(req.params.id, req.body));
})
module.exports = router
```

Iniciamos com a criação de uma instância para o roteador do Express, com o nome **router**, e outra para NotasDAO, com o nome **dao**. O tratamento de todas as rotas é similar, com a passagem dos elementos HTTP para o DAO e o retorno do resultado no formato JSON – lembrando que os métodos são assíncronos, o que irá levar à utilização de `async` e `await`.

Para o **endpoint** da **raiz**, temos o tratamento para os métodos GET e POST, em que **GET** apenas retorna todas as notas, utilizando o método **obterTodos**. Já no método **POST**, utilizamos uma chamada para **incluir**, utilizando os dados do corpo da requisição, recuperados via **req.body**.

O **endpoint** parametrizado pelo **id** tem tratamento para os métodos DELETE, GET e PUT. Enquanto os métodos **GET** e **DELETE** utilizam apenas o parâmetro da rota, recuperado via **req.params.id**, efetuando chamadas, respectivamente, para **obter** e **excluir**, o método **PUT** utiliza **alterar**, com a passagem do id e do corpo da requisição.



Destaque

Note como a estrutura do sistema está mais organizada e como as respostas para as chamadas HTTP foram implementadas de forma simples.

Com o código de roteamento completo, só precisamos criar o **servidor**, no arquivo **index.js**, com a utilização da listagem apresentada a seguir.

```
const express = require('express')
const bodyParser = require('body-parser')
const rotaNotas = require('./rota-notas')
const app = express();
app.use(bodyParser.json());
app.use('/notas', rotaNotas);
app.listen(3000, () => console.log("Servidor pronto"));
```

A configuração do servidor é extremamente simples, com a codificação JSON efetuada via **Body Parser**, e o uso das rotas de notas a partir de **/notas**. Feitas as configurações, o servidor é iniciado na porta 3000.



Para Praticar

Execute o servidor, através do comando **node index**, e teste os serviços com a ferramenta de sua preferência, como **Insomnia**, **Postman** ou **Boomerang**.

Considerações Finais da Aula

Aqui, pudemos compreender o conceito de padrão arquitetural e conhecer alguns padrões utilizados no mercado de desenvolvimento, como Event Driven, Pipes and Filters, Broker e PAC. Damos especial atenção à arquitetura MVC, que permite a divisão do sistema nas áreas de persistência, de regras de negócio e de interface de usuário.

Vimos a criação da camada Model, com base no Sequelize, e como o padrão DAO permite organizar as operações de consulta e manipulação dos dados, com a criação de uma classe que encapsula as chamadas para o Sequelize e fornece métodos específicos para gerenciamento de um tipo de entidade.

Em termos da camada Controller, utilizamos o Express para definir uma API no estilo REST, aproveitando o modelo de roteamento robusto oferecido pelo framework e a simplicidade obtida com a utilização do DAO.

Por se tratar de um sistema de back-end, não temos a camada View, mas o uso de ferramentas como Postman, Insomnia ou Boomerang permite efetuar todos os testes necessários, simulando a utilização pelo front-end.

Materiais Complementares

**Site**

Refactoring Guru

2023, Refactoring. Alexander Shvets.

O site traz informações voltadas para a melhoria de qualidade no desenvolvimento de softwares, além de oferecer um ótimo catálogo de padrões de desenvolvimento, com implementação em diversas plataformas, incluindo JavaScript.

Link para acesso: <https://refactoring.guru/> (acesso em: 13 set. 2023).

**Artigo**

Vamos falar de arquitetura de software e o modelo MVC

2023, Vitor Almeida.

O artigo descreve em detalhes a arquitetura MVC, com diversas imagens que facilitam a sua compreensão.

Link para acesso: <https://blog.grancursosonline.com.br/arquitetura-de-software-e-mvc/> (acesso em: 13 set. 2023).

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://abre.ai/gKc3>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://abre.ai/gKc6>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://abre.ai/gKdt>. Acesso em: 12 set. 2023.

ZENKER, A. **Arquitetura de sistemas**. Porto Alegre: SAGAH, 2019. Disponível em: <https://abre.ai/gKdg>. Acesso em: 13 set. 2023.